

Programming II

Object Oriented Programming **(OOP)** **Constructors**

Constructors:

- Classes can have special methods called constructors.
- A constructor is a method that is automatically called when an object is created.
- Constructors are used to perform operations at the time an object is created.
- Constructors typically initialize instance fields and perform other object initialization tasks.

Constructors:

- Constructors have a few special properties that set them apart from normal methods.
 - Constructors have the same name as the class.
 - Constructors have no return type (not even void)
 - Constructors may not return any values.
 - Constructors are typically public.

Example :

```
public class Rectangle{
    private double length;
    private double width;

    public Rectangle ()
    { length = 10;
      width = 15;
    }

    public void setlength (double l)
    {
        length = l;
    }
    public void setwidth (double w)
    {
        width = w;
    }
    public double getlength()
    {
        return length;
    }
    public double getwidth()
    {
        return width;
    }
    public double getarea()
    {
        return length*width;
    }
}
```

Example :

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle ();
        System.out.println(r1.getLength());
        System.out.println(r1.getWidth());
    }
}
```

10.0

15.0

Example :

```
public class Rectangle{
    private double length;
    private double width;
    public Rectangle ()
    { length = 10;
      width = 15;
      System.out.println("a new room created with 10 meter length and 15 meter width");
    }

    public void setlength (double l)
    {
        length = l;
    }
    public void setwidth (double w)
    {
        width = w;
    }
    public double getlength()
    {
        return length;
    }
    public double getwidth()
    {
        return width;
    }
    public double getarea()
    {
        return length*width;
    }
}
```

Example :

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle ();
        System.out.println(r1.getLength());
        System.out.println(r1.getWidth());
    }
}
```

a new room created with 10 meter length and 15 meter width

10.0

15.0

Example :

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle ();
        System.out.println(r1.getlength());
        System.out.println(r1.getwidth());
        r1.setlength(25);
        r1.setwidth(30);
        System.out.println(r1.getlength());
        System.out.println(r1.getwidth());
    }
}
```

```
a new room created with 10 meter length and 15 meter width
10.0
15.0
25.0
30.0
```


Overloading Methods and Constructors:

- Two or more methods in a class may have the same name as long as their parameter lists are different.
- When this occurs, it is called method overloading. This also applies to constructors.
- Method overloading is important because sometimes you need several different ways to perform the same operation.

Overloaded Method add:

```
public int add(int num1, int num2)
{
    int sum = num1 + num2;
    return sum;
}

public String add (String str1, String str2)
{
    String combined = str1 + str2;
    return combined;
}
```

Rectangle class Constructor Overload:

Rectangle r1 = new Rectangle ();

Rectangle r2 = new Rectangle (5, 10);

Example : (Constructor overloading)

```
public class Rectangle{  
    private double length;  
    private double width;  
    public Rectangle ()  
    { length = 10;  
      width = 15;  
    }  
    public Rectangle (double l , double w)  
    {  
        length = l;  
        width = w;  
    }  
}
```

Without Any Arguments

With two Arguments

```
public class Main  
{  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle ();  
        System.out.println(r1.getLength());  
        System.out.println(r1.getWidth());  
        System.out.println();  
        Rectangle r2 = new Rectangle (40 , 60);  
        System.out.println(r2.getLength());  
        System.out.println(r2.getWidth());  
    }  
}
```

10.0

15.0

40.0

60.0

The Default Constructor:

- When an object is created, its constructor is always called.
- If you do not write a constructor, Java provides one when the class is compiled. The constructor that java provides is known as the default constructor.
 - It sets all of the object's numeric fields to 0.
 - It sets all of the object's Boolean fields to false.
 - It sets all of the object's reference variables to the special value null.

The Employee Example:

<i>Employee</i>
Emp_ID : Int Name : String Depart : String Salary : Double Bonus : Double Resident : Boolean
Employee () Employee (Int, String) Employee (int , String , bool) Employee (int , String, String , double ,double, bool) Set_Salary (double): Void Set_Salary (double , double) : Void Set_depart (String): Void Set_Emp_ID (int): void Print_Emp_Data : Void

Example :

```
public class Employee{
    private int emp_id;
    private String ename;
    private String depart;
    private double salary;
    private double bonous;
    private boolean resident;
    public void print_emp_data()
    { System.out.println("ID "+emp_id);
      System.out.println("Name "+ename);
      System.out.println("depart "+depart);
      System.out.println("Salary "+salary);
      System.out.println("Bonous "+bonous);
      System.out.println("Resident "+resident);
    }
}
```

Example : (Default Constructor)

```
public class Main
{
    public static void main(String[] args) {
        Employee e1 = new Employee ();
        e1.print_emp_data();
    }
}
```

```
ID 0
Name null
depart null
Salary 0.0
Bonous 0.0
Resident false
```


Example : (Constructor without any arguments)

```
public class Employee{
    private int emp_id;
    private String ename;
    private String depart;
    private double salary;
    private double bonous;
    private boolean resident;
    public Employee()
    {
        emp_id = 200;
        ename = "No Name";
        depart = "Not Assigned Yet";
        salary = 2000;
        bonous = 300;
        resident = true;
    }
    public void print_emp_data()
    { System.out.println("ID "+emp_id);
      System.out.println("Name "+ename);
      System.out.println("depart "+depart);
      System.out.println("Salary "+salary);
      System.out.println("Bonous "+bonous);
      System.out.println("Resident "+resident);
    }
}
```

Example : (Constructor without any arguments)

```
public class Main
{
    public static void main(String[] args) {
        Employee e1 = new Employee ();
        e1.print_emp_data();
    }
}
```

```
ID 200
Name No Name
depart Not Assigned Yet
Salary 2000.0
Bonous 300.0
Resident true
```

Example : (Constructor Overloading)

```
public class Employee{
    private int emp_id;
    private String ename;
    private String depart;
    private double salary;
    private double bonous;
    private boolean resident;
    public Employee()
    {
        emp_id = 200;
        ename = "No Name";
        depart = "Not Assigned Yet";
        salary = 2000;
        bonous = 300;
        resident = true;
    }
    public Employee (int idno , String n)
    {
        emp_id = idno;
        ename = n;
    }
    public Employee (int idno , String n , boolean r)
    {
        emp_id = idno;
        ename = n;
        resident = r;
    }
}
```

Example : (Constructor Overloading)

```
public class Main
{
    public static void main(String[] args) {

        Employee e1 = new Employee (300, "Mahmoud" , true);
        e1.print_emp_data();
    }
}
```

```
ID 300
Name Mahmoud
depart null
Salary 0.0
Bonous 0.0
Resident true
```